



Ansible Playbook条件语句

誉天教育

- when条件语句的用法
- 比较运算符
- 条件运算符
- 条件判断
- block , resuce和always
- 条件判断与错误处理

- 在有的时候play的结果依赖于变量、fact或者是前一个任务的执行结果，或者基于上一个task执行返回的结果而决定如何执行后续的task，这个时候就需要用到条件判断。
- 条件语句在Ansible中的使用场景：
 - ◆ 在目标主机上定义了一个硬限制，比如目标主机的最小内存必须达到多少，才能执行该task
 - ◆ 捕获一个命令的输出，根据命令输出结果的不同以触发不同的task
 - ◆ 根据不同目标主机的facts，以定义不同的task
 - ◆ 根据目标机的cpu的大小，以调优相关应用性能
 - ◆ 用于判断某个服务的配置文件是否发生变更，以确定是否需要重启服务

- 在ansible中，使用条件判断的关键字就是when。如在安装包的时候，需要指定主机的操作系统类型，或者是当操作系统的硬盘满了之后，需要清空文件等，可以使用when语句来做判断。当表达式的结果返回的是false，便会跳过本次的任务。

- 下面是一个基本的用法示例：

- name: Install vim

hosts: all

tasks:

- name: Install VIM via yum

yum:

name: vim-enhanced

state: installed

when: ansible_os_family == "RedHat"

- name: Install VIM via apt

apt:

name: vim

state: installed

when: ansible_os_family == "Debian"

- name: Unexpected OS family

debug: msg="OS Family {{ ansible_os_family }} is not supported"

when: not ansible_os_family == "RedHat" or ansible_os_family == "Debian"

- 在ansible中，还支持如下比较运算符：
 - ◆ ==：比较两个对象是否相等，相等则返回真。可用于比较字符串和数字
 - ◆ !=：比较两个对象是否不等，不等则为真。
 - ◆ >：比较两个对象的大小，左边的值大于右边的值，则为真
 - ◆ <：比较两个对象的大小，左边的值小于右边的值，则为真
 - ◆ >=：比较两个对象的大小，左边的值大于等于右边的值，则为真
 - ◆ <=：比较两个对象的大小，左边的值小于等于右边的值，则为真
- 示例：
 - ◆ when: ansible_machine == "x86_64"
 - ◆ when: max_memory <= 512

- 在Ansible中，除了比较运算符，还支持逻辑运算符：
 - ◆ and：逻辑与，当左边和右边两个表达式同时为真，则返回真
 - ◆ or：逻辑或，当左右和右边两个表达式任意一个为真，则返回真
 - ◆ not：逻辑否，对表达式取反
 - ◆ ()：当一组表达式组合在一起，形成一个更大的表达式，组合内的所有表达式都是逻辑与的关系

- 示例：

逻辑或

when: ansible_distribution == "RedHat" or ansible_distribution == "Fedora"

逻辑与

when: ansible_distribution_version == "8.0" and ansible_kernel == "4.18.0-80.el8.x86_64 "

组合

when: =>

(ansible_distribution == "RedHat" and ansible_distribution_major_version == "8")

or

(ansible_distribution == "Fedora" and ansible_distribution_major_version == "28")

- 完整的示例，判断register注册变量的返回结果
 - name: restart httpd if postfix is running
 - hosts: test
 - tasks:
 - name: get postfix server status
 - command: /usr/bin/systemctl is-active postfix
 - ignore_errors: yes
 - register: result
 - name: restart apache httpd based on postfix status
 - service:
 - name: httpd
 - state: restarted
 - when: result.rc == 0

- 在shell当中，我们可使用test命令来进行一些常用的判断操作，如下：

判断/test文件是否存在

```
test -e /test
```

判断/testdir是否存在且为一个目录

```
test -d /testdir
```

- 事实上，在ansible中也有类似的用法，下面是一个简单示例：

通过条件语句判断testpath的路径是否存在

```
- hosts: test
```

```
vars:
```

```
testpath: /testdir
```

```
tasks:
```

```
- debug:
```

```
  msg: "file exist"
```

```
  when: testpath is exists
```

- 上面的示例中，我们使用了is exists用于路径存在时返回真，也可以使用is not exists用于路径不存在时返回真。也可以在整个条件表达式的前面使用not以取反：

```
- hosts: test
```

```
vars:
```

```
testpath: /testdir1
```

```
tasks:
```

```
- debug:
```

```
  msg: "file not exist"
```

```
  when: not testpath is exists
```

- 在ansible中，除了能够使用exists这种tests之外，还有一些别的tests。接下来我们详细说一说。

- 判断变量：
 - ◆ defined：判断变量是否已定义，已定义则返回真
 - ◆ undefined：判断变量是否未定义，未定义则返回真
 - ◆ none：判断变量的值是否为空，如果变量已定义且值为空，则返回真

- 示例。

```
- hosts: test
gather_facts: no
```

```
vars:
```

```
  testvar: "test"
```

```
  testvar1:
```

```
tasks:
```

```
  - debug:
```

```
    msg: "testvar is defined"
```

```
    when: testvar is defined
```

```
- debug:
```

```
  msg: "testvar2 is undefined"
```

```
  when: testvar2 is undefined
```

```
- debug:
```

```
  msg: "testvar1 is none"
```

```
  when: testvar1 is none
```

- 判断执行结果：

- ◆ ok：目标状态与期望值一致，没有发生变更
- ◆ change或changed：目标发生变更，与期望值一样
- ◆ sucess或succeeded：目标状态与期望值一致，或者任务执行成功
- ◆ failure或failed：任务执行失败
- ◆ skip或skipped：任务被跳过

- 示例：

- hosts: test

- tasks:

- shell: 'cat /testdir/aaa'

- register: result

- ignore_errors: true

- debug:

- msg: "success"

- when: result is success

- debug:

- msg: "failed"

- when: result is failure

- debug:

- msg: "changed"

- when: result is change

- debug:

- msg: "skip"

- when: result is skip

- 判断路径：
 - ◆ file：判断指定路径是否为一个文件，是则为真
 - ◆ directory：判断指定路径是否为一个目录，是则为真
 - ◆ link：判断指定路径是否为一个软链接，是则为真
 - ◆ mount：判断指定路径是否为一个挂载点，是则为真
 - ◆ exists：判断指定路径是否存在，存在则为真
 - ◆ 特别注意：关于路径的所有判断均是判断主控端上的路径，而非被控端上的路径

- 示例：

```
- hosts: test
gather_facts: no
vars:
```

```
testpath1: "/testdir/test"
testpath2: "/testdir"
```

```
tasks:
```

```
- debug:
  msg: "file"
  when: testpath1 is file
```

```
- debug:
  msg: "directory"
  when: testpath2 is directory
```

- 判断字符串：
 - ◆ lower：判断字符串中的所有字母是否都是小写，是则为真
 - ◆ upper：判断字符串中的所有字母是否都是大写，是则为真

- 示例：

```
- hosts: test
gather_facts: no
vars:
  str1: "abc"
  str2: "ABC"
tasks:
  - debug:
      msg: "str1 is all lowercase"
    when: str1 is lower
  - debug:
      msg: "str2 is all uppercase"
    when: str2 is upper
```

- 判断整除：
 - ◆ even：判断数值是否为偶数，是则为真
 - ◆ odd：判断数值是否为奇数，是则为真
 - ◆ divisibleby(num)：判断是否可以整除指定的数值，是则为真
- 示例：
 - hosts: test
 - gather_facts: no
 - vars:
 - num1: 6
 - num2: 8
 - num3: 15
 - tasks:
 - debug:
 - msg: "num1 is an even number"
 - when: num1 is even
 - debug:
 - msg: "num2 is an odd number"
 - when: num2 is odd
 - debug:
 - msg: "num3 can be divided exactly by"
 - when: num3 is divisibleby(3)

- 其他条件测试：
 - ◆ subset 判断一个list是不是另一个list的子集：when: a is subset(b)
 - ◆ superset 判断一个list是不是另一个list的父集：when: b is superset(a)
 - ◆ in 判断一个字符串是否存在于另一个字符串中，也可用于判断某个特定的值是否存在于列表中
supported_distros:
 - RedHat
 - CentOSwhen: ansible_distribution in supported_distros
 - ◆ string 判断对象是否为一个字符串，是则为真 when: var1 is string
 - ◆ number 判断对象是否为一个数字，是则为真 when: var3 is number

- 我们在前面使用when做条件判断时，如果条件成立则执行对应的任务。但这就面临一个问题，当我们要使用同一个条件判断执行多个任务的时候，就意味着我们要在某一个任务下面都写一下when语句，而且判断条件完全一样。这种方式不仅麻烦而且显得low。Ansible提供了一种更好的方式来解决这个问题，即block。
- 在ansible中，使用block将多个任务进行组合，当作一个整体。我们可以对这一个整体做条件判断，当条件成立时，则执行块中的所有任务：

- 示例：

- hosts: node1

- tasks:

- block:

- name: install apache

- yum:

- name: httpd

- state: present

- name: started apache

- service:

- name: httpd

- state: started

- enabled: yes

- when: ansible_os_family == "RedHat"

- block除了能和when一起使用之外，还能作错误处理。这个时候就需要用到rescue关键字：

- hosts: test

- tasks:

- block:

- shell: 'ls /testdir'

- rescue:

- debug:

- msg: '/testdir is not exists'

- 当block执行失败时，rescue中的任务才会被执行；而无论block执行成功还是失败，always中的任务都会被执行：

- hosts: test

- tasks:

- block:

- shell: 'ls /testdir'

- rescue:

- debug:

- msg: '/testdir is not exists'

- always:

- debug:

- msg: 'This task always executes'

- 在shell中，可能会有这样的需求：当脚本执行至某个阶段时，需要对某个条件进行判断，如果条件成立，则立即终止脚本的运行。在shell中，可以直接调用"exit"即可执行退出。事实上，在playbook中也有类似的模块可以做这件事。即fail模块。
- fail模块用于终止当前playbook的执行，通常与条件语句组合使用，当满足条件时，终止当前play的运行。选项只有一个：msg：终止前打印出信息

使用fail模块中断playbook输出

- hosts: test

tasks:

- shell: echo "Just a test--error"

register: result

- fail:

msg: "Conditions established,Interrupt running playbook"

when: "'error' in result.stdout"

- debug:

msg: "I never execute,Because the playbook has stopped"

- 事实上，当fail和when组合使用的时候，还有一个更简单的写法，即failed_when，当满足某个条件时，ansible主动触发失败。
- ansible一旦执行返回失败，后续操作就会中止，所以failed_when通常可以用于满足某种条件时主动中止playbook运行的一种方式。

如果在command_result存在错误输出，且错误输出中，包含了`FAILED`字符串，即返回失败状态：

- hosts: test

tasks:

- name: this command prints FAILED when it fails

command: /usr/bin/example-command -x -y -z

register: command_result

failed_when: "'FAILED' in command_result.stderr"

- ✓ when条件语句的用法
- ✓ 比较运算符
- ✓ 条件运算符
- ✓ 条件判断
- ✓ block , resuce和always
- ✓ 条件判断与错误处理

